

- ARM RTOS 운영체제 이해
 - ARM RTOS 운영체제 개념 학습
- RTOS API 활용 방법
 - RTOS API를 활용한 Thread, Mutex, Semaphore 등 사용방법

- 본 학습 내용으로 들어가기 전, 학습 주제의 흥미를 이끌 만한 도입부의 내용이 있다면 제시해주세요.
- ex. 관련 뉴스기사, 실생활과 관련된 이야기 등
- 저작권 침해가 되지 않도록 내용을 구성해 주세요.
- 출처가 있을 경우 반드시 작성해 주세요.

STM32 Nucleo F103RB 보드는 1~2만원대 매우 저렴한 가격으로 시중에서 쉽게 구할 수 있고, ST-LINK 디버거를 포함하고 있어 ARM 임베디드 프로그래밍을 학습하기에 적합한 개발보드입니다.

이러한 보드에서 실행되는 ARM 명령어는 크게 데이터 전송 명령어, 데이터 처리 명령어, 제어 흐름 명령어로 분류됩니다. 우리는 각 명령어 분류에 대한 대략적인 개념과 명령어 예시를 통해 ARM 프로세서의 이론적 배경을 학습합니다.

뿐만 아니라, STM32 실습보드에서 활용가능한 통합개발환경을 소개하고, 널리 사용되는 통합개발환경을 활용하여 STM32 보드에서 GPIO를 제어하는 임베디드 소프트웨어를 실행하는 방법을 학습하도록 하겠습니다.

◆ 학습목표

1. ARM RTOS 운영체제 이해
2. RTOS API 활용 방법

◆ 학습내용

1. ARM RTOS 운영체제 개념 학습
2. RTOS API를 활용한 Thread, Mutex, Semaphore 등 사용방법

- 학습내용과 학습목표는 강의계획서와 일치해야 하며, 필요시 강의계획서를 수정할 수 있습니다.
- **학습목표**
 - ✓ 각 레슨에 맞는 학습목표를 2~3개 작성해 주세요.
- ③ **학습내용**
 - ✓ 1회차 당 25분 분량이 되도록 2~3개 레슨으로 구성해주세요.
 - ✓ 학습내용과 레슨명은 일치해야 합니다.

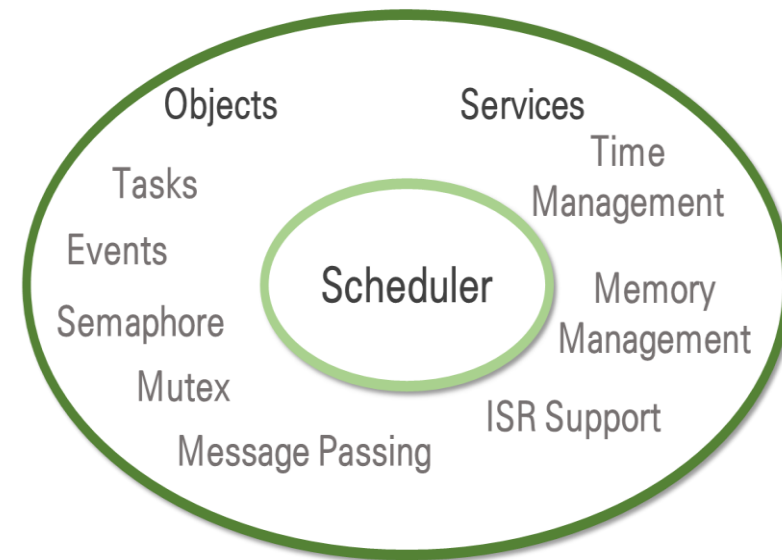
간지

ARM RTOS 운영체제 개념 이해

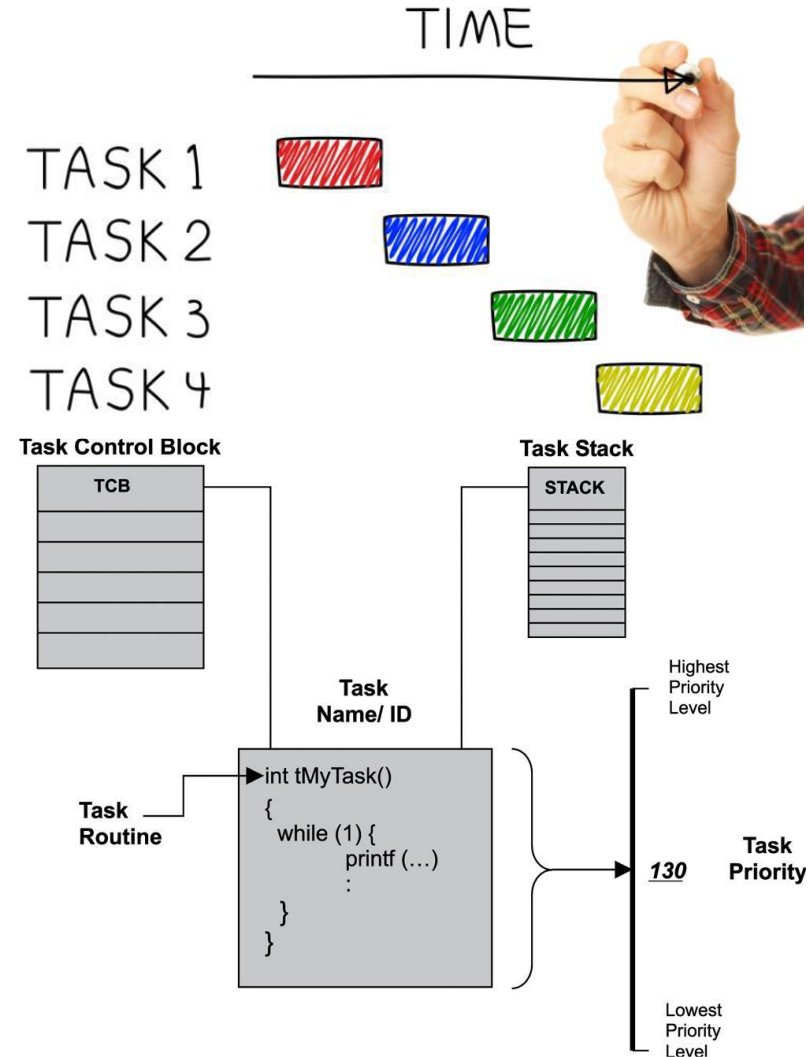
- ARM 프로세서에서 RTOS 탑재 및 활용
- RTOS
- 실시간성(Real Time)
설계자가 정한 시간 내에 태스크(작업) 처리 또는 실행이 완료되어야 하는 특성
- 실시간 운영체제(Real Time Operating System)
Real Time 성능을 보장하는 Operating System

- ARM 프로세서에서 RTOS 탑재 및 활용
- RTOS
- OS specific features
 - Multi-Tasking 지원
 - Preemptive scheduling
- Embedded system features
 - Low memory footprint
 - Fast execution : Exclude general purpose & features
 - Scalable : ROM/RAM size & OS services
 - Portable : C language
- Objects
 - Task, Semaphore, Mutex, Message Queue, Event, ISR
- Services
 - Timer process, Interrupt Process, Resource management

- ARM 프로세서에서 RTOS 탑재 및 활용
- CMSIS-RTOS
- 프로그램 스레드의 round robin, preemptive and cooperative multitasking을 지원하는 스케줄러와 time and memory management 서비스로 구성됨.
- Task
 - Multiple tasks with the application can be implemented by separate threads
 - Defined by Task Control Block(TCB)
- Semaphore
- Mutex
- Message Queue
- Message Mailbox
- Events
- Timers
- ISR



- ARM 프로세서에서 RTOS 탑재 및 활용
- Task Object
- Concurrent task design 필요
 - 일련의 개별 동작을 순차적으로 처리하도록 만든 응용 프로그램은 실시간 성능을 만족하기 어려움
 - 실시간 성능을 만족하기 위하여 스케줄링 가능한 작고 프로그램 단위로 분할
- 응용 프로그램을 여러 개의 태스크로 분류
 - 정해진 시간제약 안에서 최적화된 입출력 작업 처리를 위함
- 태스크 오브젝트(Task Object)
 - 이름, ID, 우선순위, TCB(Task Control Block), 스택, 태스크 루틴



Cortex-M3 ARM 임베디드 시스템 설계 입문

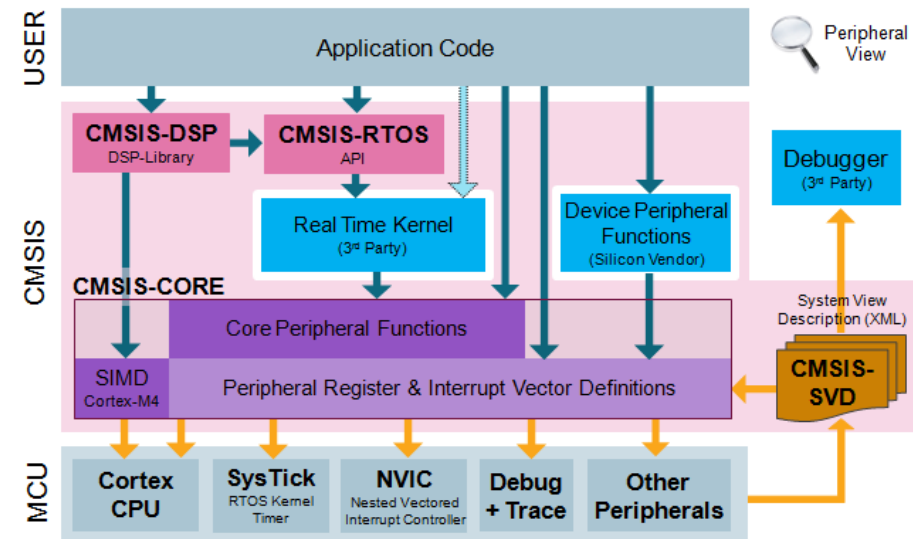
- ARM 프로세서에서 RTOS 탑재 및 활용
- CMSIS
- 애플리케이션 코드에서 CMSIS-RTOS 기능에 액세스하려면 다음 헤더 파일을 포함해야 함
- `#include <cmsis_os.h>`
- 이 헤더 파일은 ARM에서 CMSIS-RTOS 표준의 일부로 관리합니다. CMSIS-RTOS Keil RTX의 경우 이 헤더 파일이 기본 API임

Cortex/Common Microcontroller Software Interface Standard

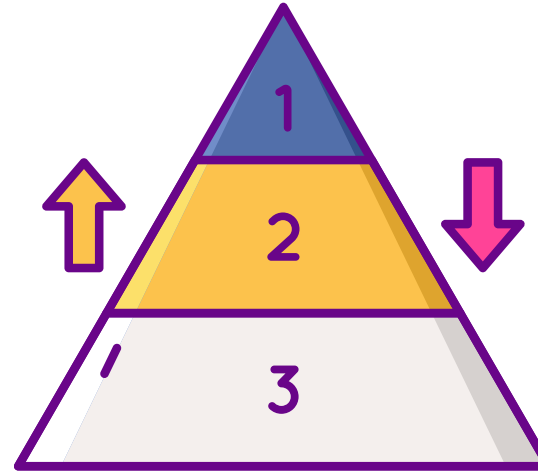
Vendor independent hardware abstraction layer for Cortex M series processors

Consists of CORE, DSP, RTOS API, and SVD.

ARM 프로세서에서 RTOS 탑재 및 활용



- ARM 프로세서에서 RTOS 탑재 및 활용
- CMSIS-RTOS Priority Levels
 - osPriorityIdle
 - osPriorityLow
 - osPriorityBelowNormal
 - osPriorityNormal
 - osPriorityAboveNormal
 - osPriorityHigh
 - osPriorityRealTime
 - osPriorityError



- ARM 프로세서에서 RTOS 탑재 및 활용
- Open-CMSIS-Pack

Cortex/Common Microcontroller Software Interface Standard

Vendor independent hardware abstraction layer for Cortex M series processors

Consists of CORE, DSP, RTOS API, and SVD.

소프트웨어 구성 요소, 장치 매개변수 및 평가 보드 지원을 위한 전달 메커니즘을 설명

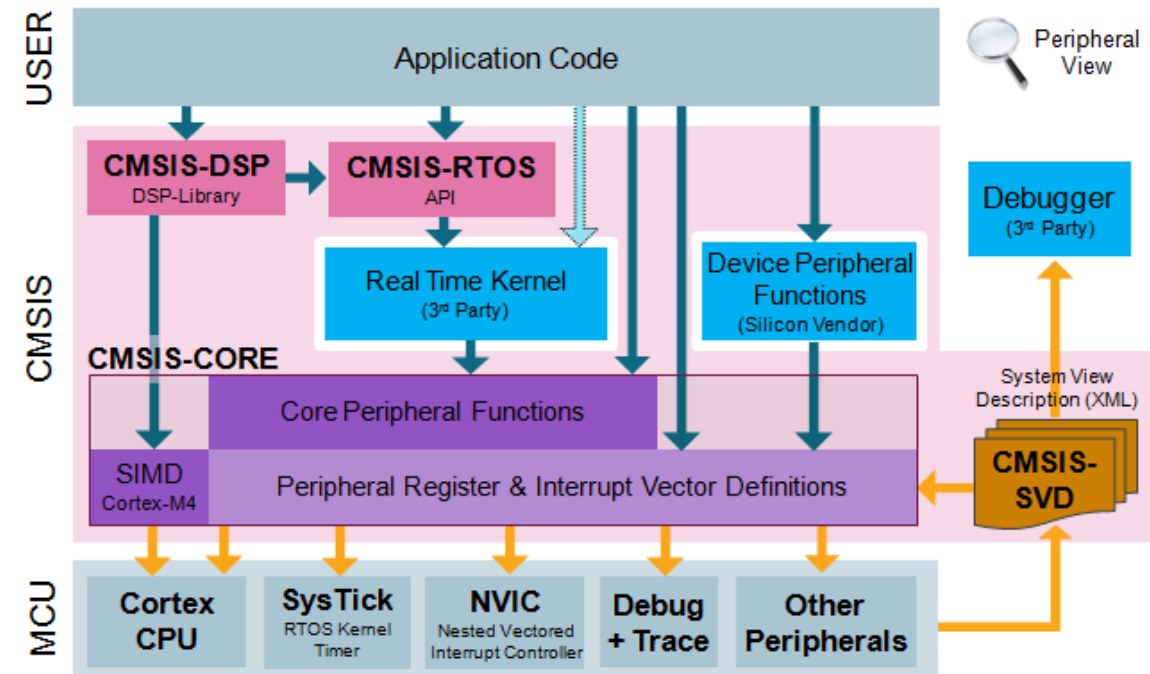
XML 기반 패키지 설명(PDSC) 파일은 다음을 포함하는 software pack(파일 컬렉션)의 내용을 설명

소스 코드, 헤더 파일 및 소프트웨어 라이브러리, 문서 및 소스 코드 템플릿

Device parameter와 startup code 및 프로그래밍 알고리즘

example projects

ARM은 칩 벤더 회사를 위해 장치 지원, 프로세서, 주변 장치에 대한 consistent software interface 를 가능하도록 하는 CMSIS소스 코드 제공



- ARM 프로세서에서 RTOS 탑재 및 활용
- Open-CMSIS-Pack

Software Pack(file collection) 사용 사례 : 소프트웨어 팩을 사용하여 다음을 제공할 수 있음.

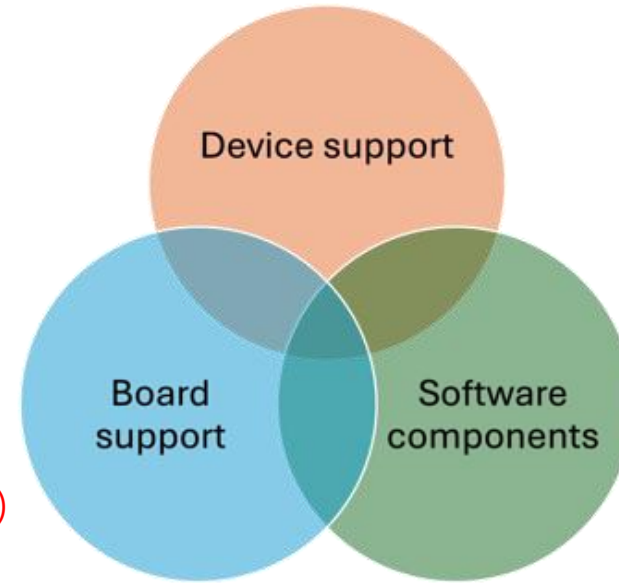
Device support : 이온바 디바이스 패밀리 팩(DFP)은 마이크로컨트롤러 디바이스 패밀리를 위한 CMSIS 시스템/스타트업 파일, 드라이버 및 플래시 알고리즘을 포함.

Board support : 보드 지원 팩(BSP)에는 특정 개발 보드에 대한 문서, 도식 및 드라이버가 포함되어 있음.

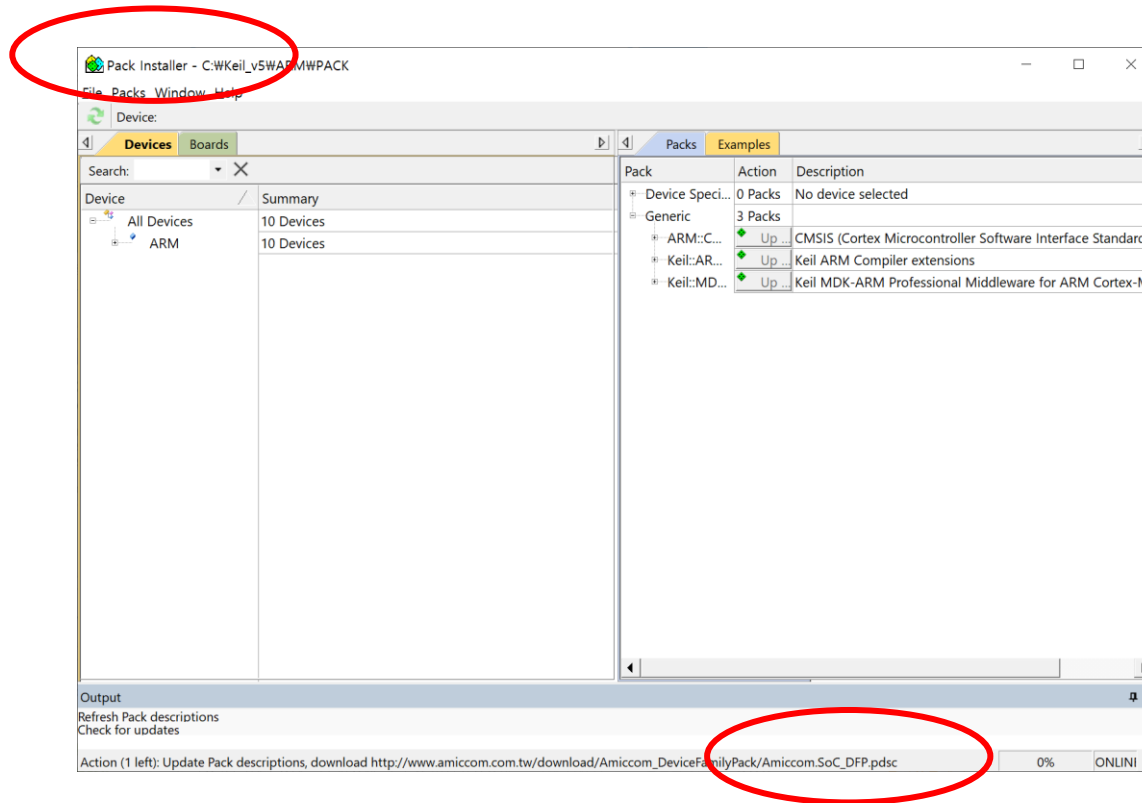
Software components : 순수 소프트웨어 팩(Software Pack)에는 소스 코드, 라이브러리, 헤더파일, 문서 등이 포함될 수 있음

공용으로 사용할 수 있는 미들웨어 스택.

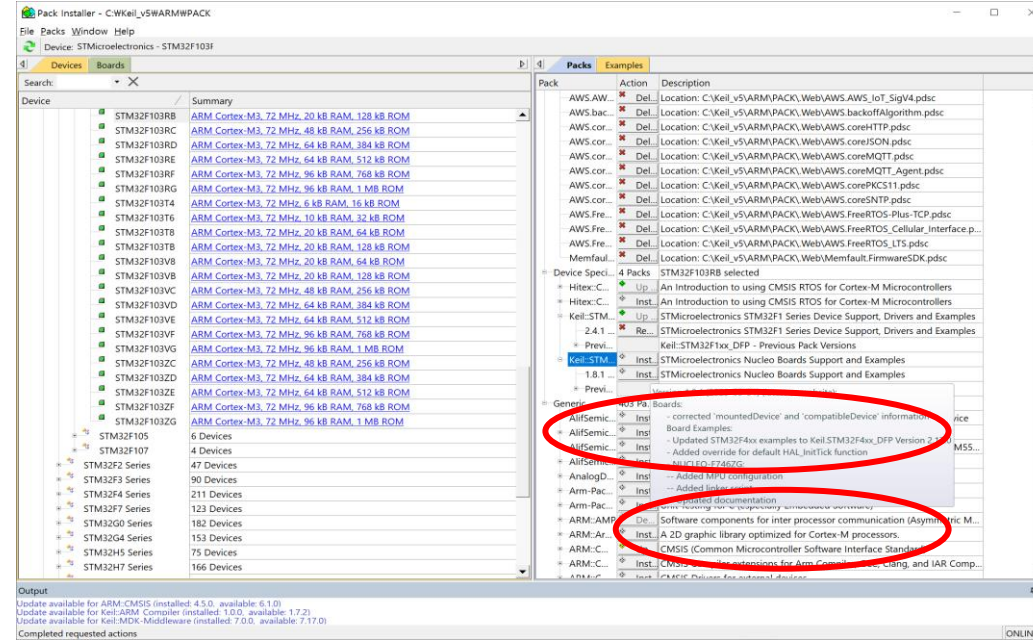
회사 및/또는 엔지니어링 그룹 내 배포를 위한 사내 소프트웨어.



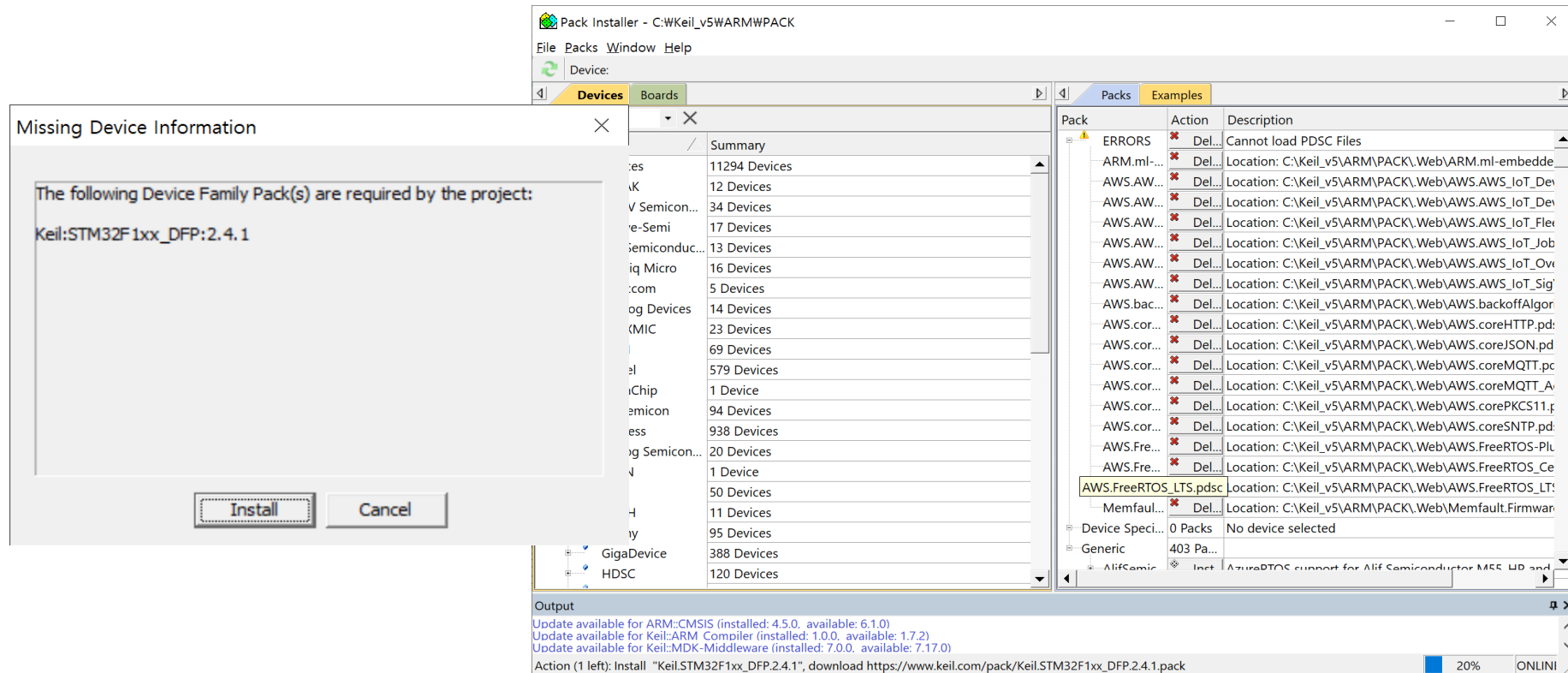
- ARM 프로세서에서 RTOS 탑재 및 활용
- XML 기반 패키지 설명(PDSC) 파일은 다음과 같은 Software pack(파일 컬렉션)의 내용을 describe함.
 - Device support
 - Board support
 - Software components



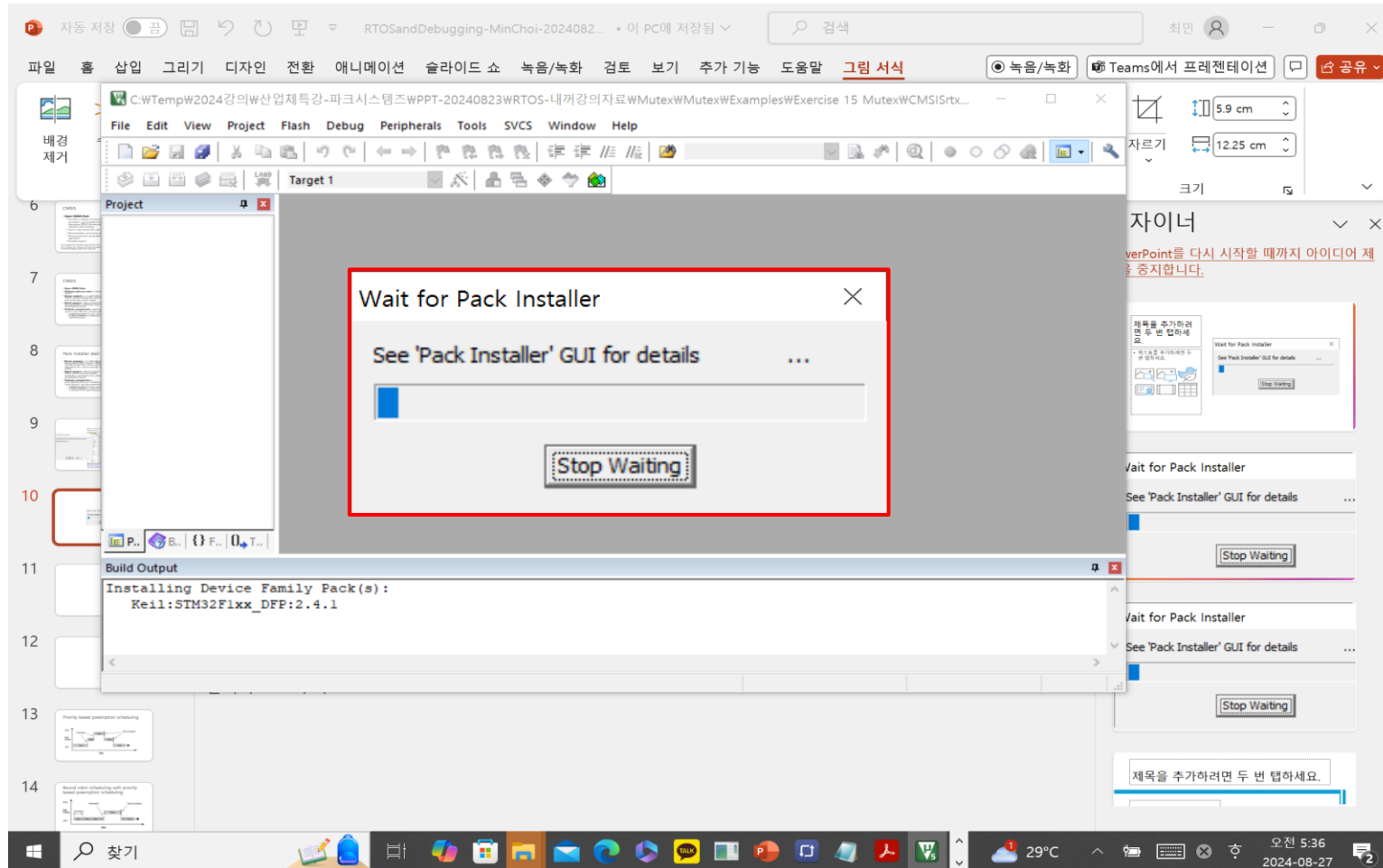
- ARM 프로세서에서 RTOS 탑재 및 활용
- XML 기반 패키지 설명(PDSC) 파일은 다음과 같은 Software pack(파일 컬렉션)의 내용을 describe함.
Device support
Board support
Software components



- ARM 프로세서에서 RTOS 탑재 및 활용



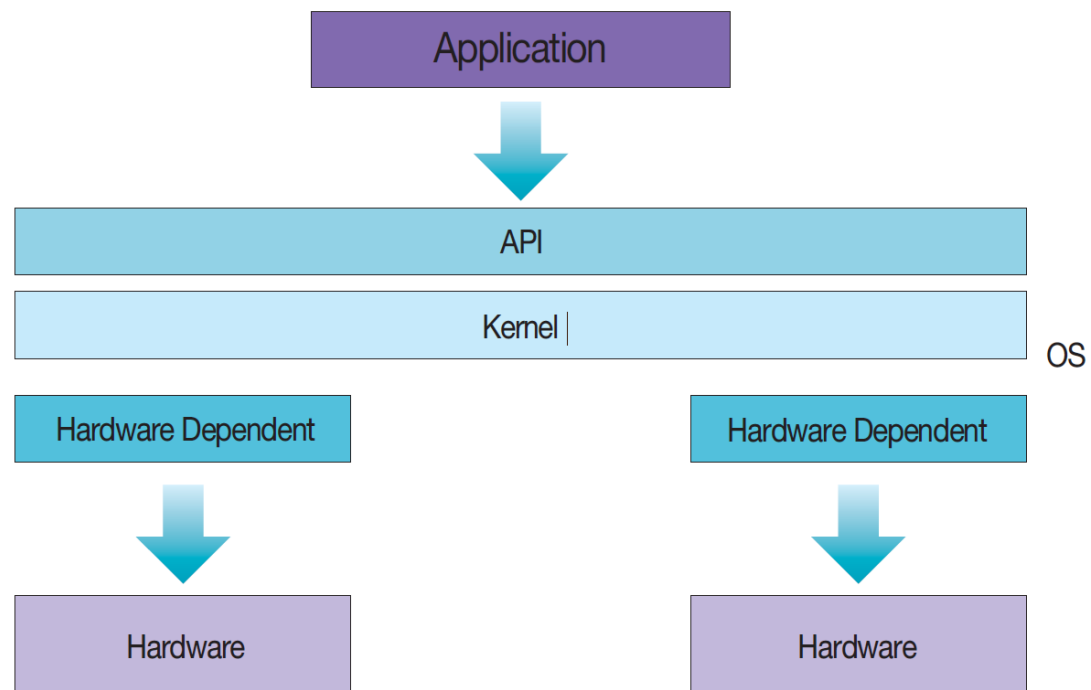
- ARM 프로세서에서 RTOS 탑재 및 활용



- ARM 프로세서에서 RTOS 탑재 및 활용
- RTOS 쓰레드

```
17 int main (void) {  
18  
19     SystemInit(); //system initialization  
20  
21     GPIO_SetDir(1, 0xB0000000, 1); //Set 28,29,31 of GPIO1 as output  
22     GPIO_SetDir(2, 0x0000007C, 1); //Set 2,3,4,5,6 of GPIO2 as output  
23  
24     GPIO_ClearValue(1, 0xB0000000); //Set 28,29,31 of GPIO1 off : LED1,2,3 off  
25     GPIO_ClearValue(2, 0x0000007C); //Set 2,3,4,5,6 of GPIO2 off : LED4,5,6,7,8 off  
26  
27     osKernelInitialize ();           // initialize CMSIS-RTOS  
28     Init_Thread ();  
29     osKernelStart ();               // start thread execution  
30  
31 }
```

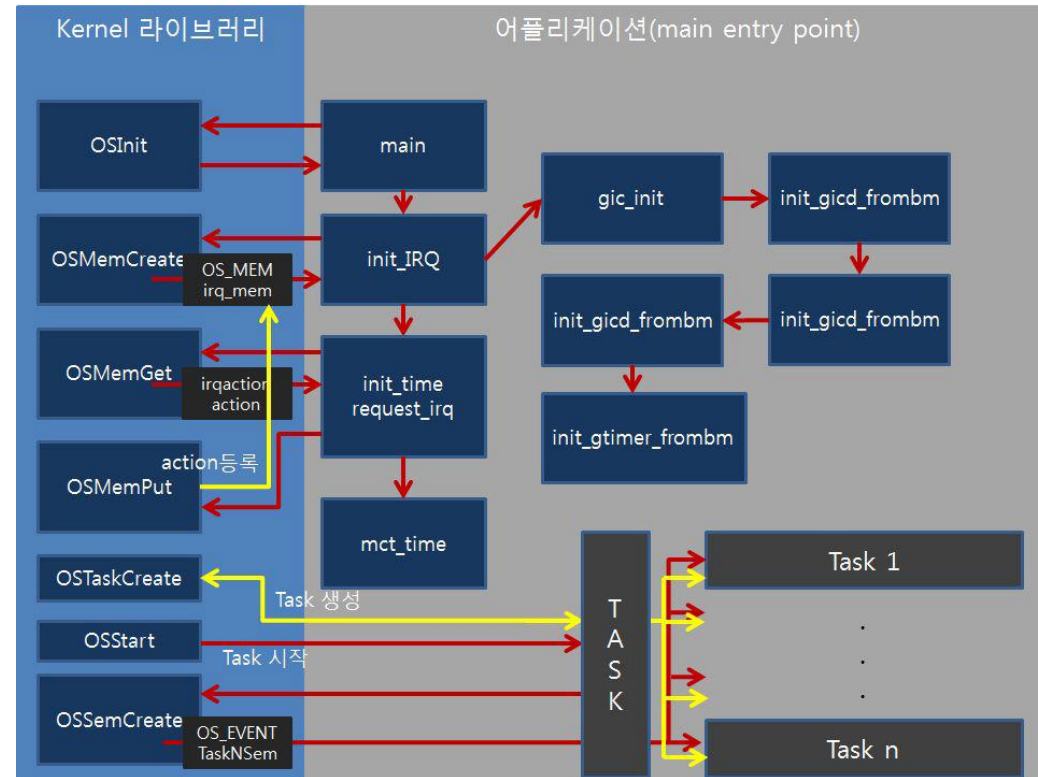

- ARM 프로세서에서 RTOS 탑재 및 활용
- uC/OS-II 및 FreeRTOS 구조



Cortex-M3 ARM 임베디드 시스템 설계 입문

- ARM 프로세서에서 RTOS 탑재 및 활용
- uC/OS-II 소스코드 구조
- Developed by Jean J. Labrosse
- – 선점형(preemptive), 태스크 관리, 이벤트 관리 (semaphore, mutex, mailbox, etc)
- – C코드 8000 라인, 일부 어셈블리(asm) 코드

ARM 프로세서에서 RTOS 탑재 및 활용



- ARM 프로세서에서 RTOS 탑재 및 활용
- uC/OS-II 소스코드 구조
- 특성

Multitasking : 각 Task는 영향을 받지 않고, 독립적으로 수행, 시퀀스 프로그램에서 main 함수 3개가 동시에 수행되는 것처럼 동작

Preemptive (선점형 커널)

Deterministic (실행시간 예측가능)

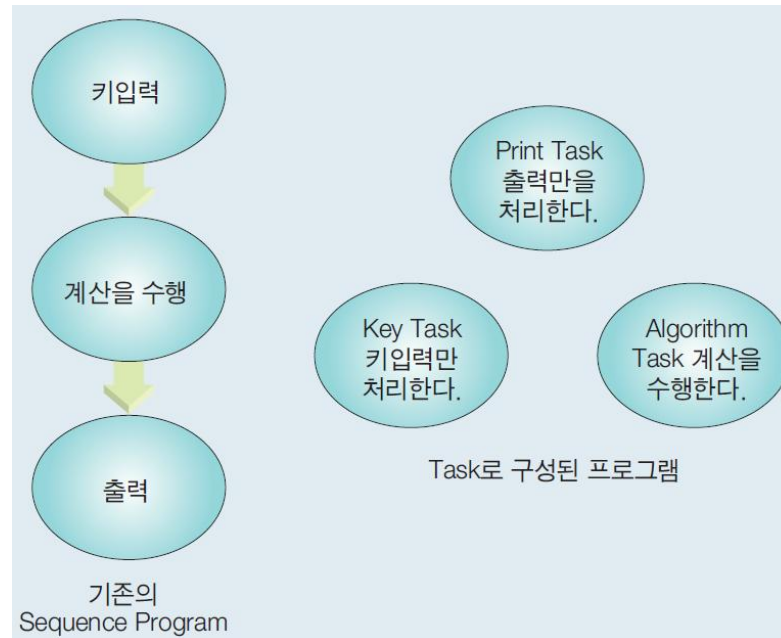
- 장점

CPU utilization/efficiency 개선

분할 구조 가능

- 단점

Scheduling overhead, task간 상호작용 예측 어려움,



- ARM 프로세서에서 RTOS 탑재 및 활용
- uC/OS-II 소스코드 구조

```
#include "includes.h"
#define TASK_STK_SIZE 512
#define N_TASKS 2
OS_STK TaskStk[N_TASKS][TASK_STK_SIZE]; ----- (1)
OS_STK TaskStartStk[TASK_STK_SIZE]; ----- (2)
void Task1(void *data);
void Task2(void *data);
void TaskStart(void *data);
void main (void)
{
    PC_DispcrScr(DISP_FGND_WHITE + DISP_BGND_BLACK);
    OSInit();
    PC_DOSaveReturn();
    PC_VectSet(uCOS, OSCtxSw);
    OSTaskCreate(TaskStart, (void *)0, (void *)
    *)&TaskStartStk[TASK_STK_SIZE - 1], 0);
    OSStart( );
}
void TaskStart (void *data)
{
    UBYTE i;
    char s[100];
    WORD key;
    data = data
    OS_ENTER_CRITICAL();
    PC_VectSet(0x08, OSTickISR);
    PC_SetTickRate(OS_TICKS_PER_SEC);
    OS_EXIT_CRITICAL();
```

[함수 원형]

int8u OSTaskCreate(void (*task)(void
*pd), void *pdata, OSSTK *ptos, int8u
prio)

[argument]

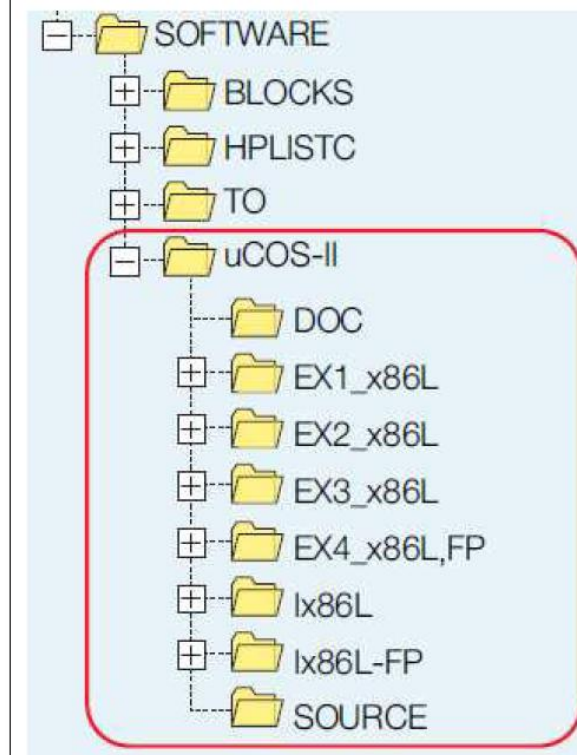
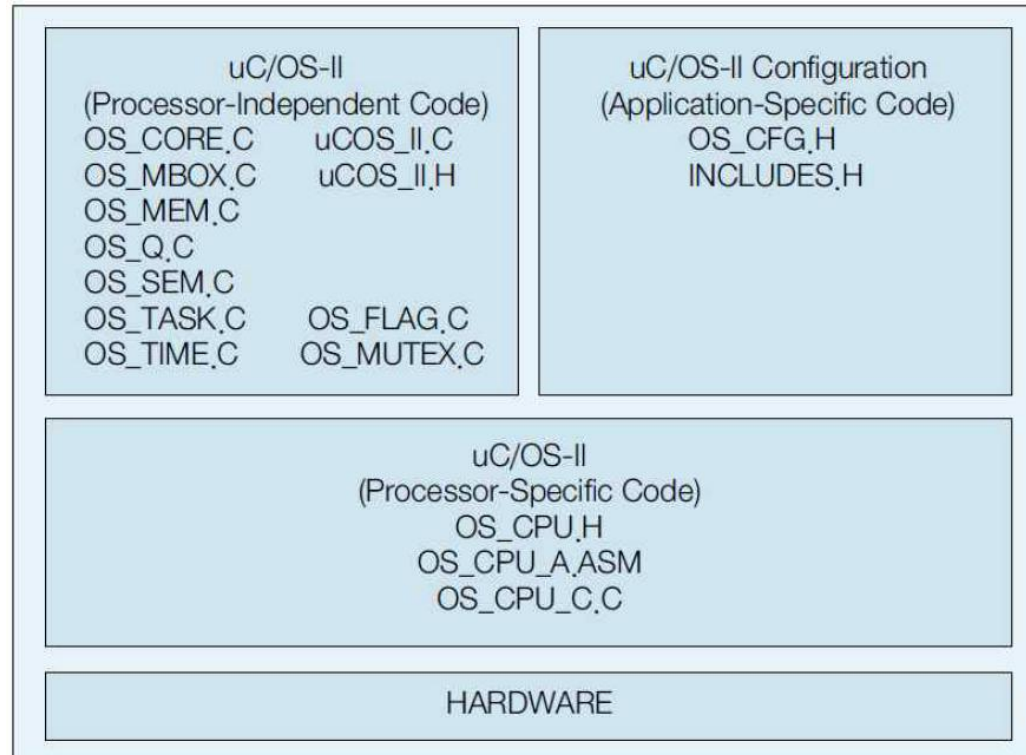
task - Task로 만들기 위한 함수의 시작
주소

pdata - Task에게 넘길 인자

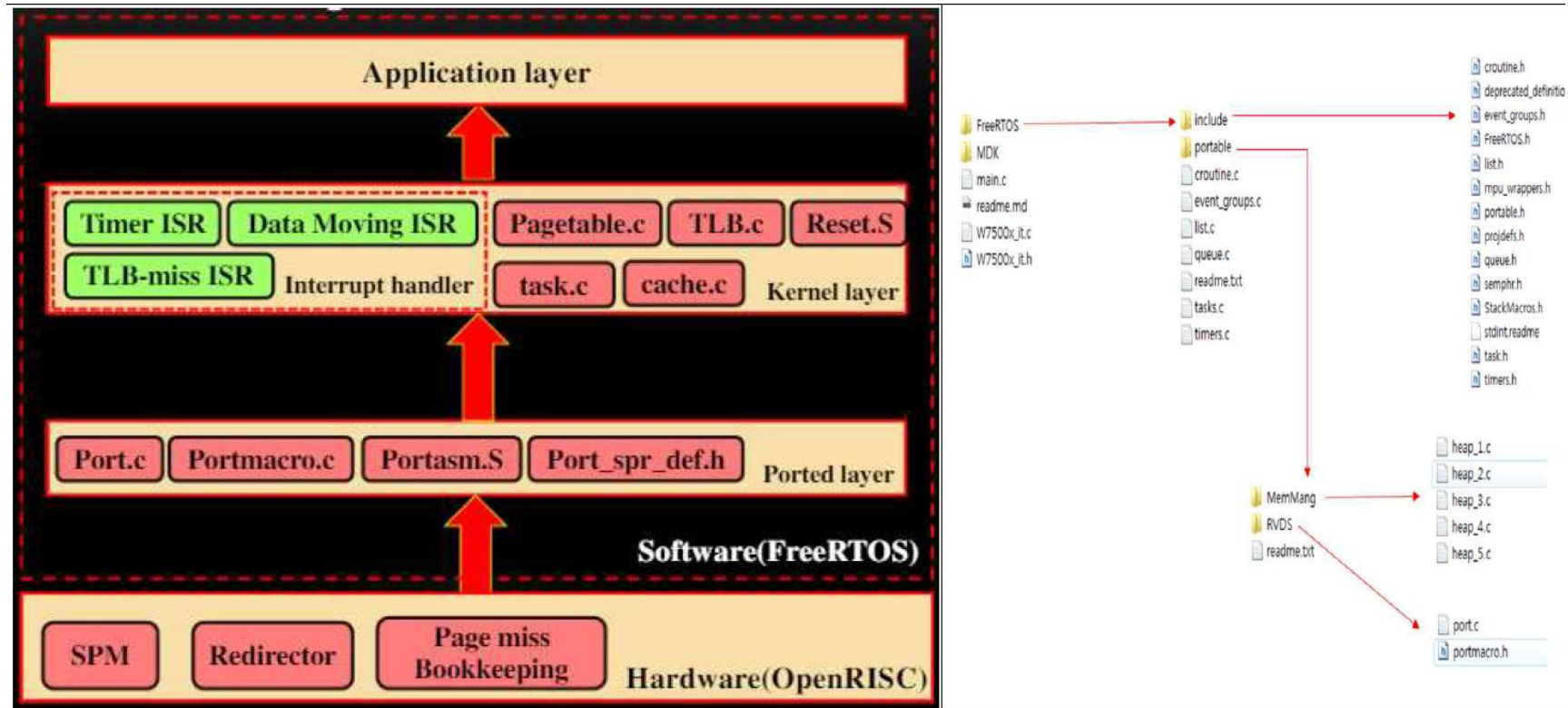
ptos - Task가 사용할 스택의 시작주소

prio - Task의 우선순위

- ARM 프로세서에서 RTOS 탑재 및 활용
- uC/OS-II 소스코드 구조



- ARM 프로세서에서 RTOS 탑재 및 활용
- FreeRTOS 소스코드 구조



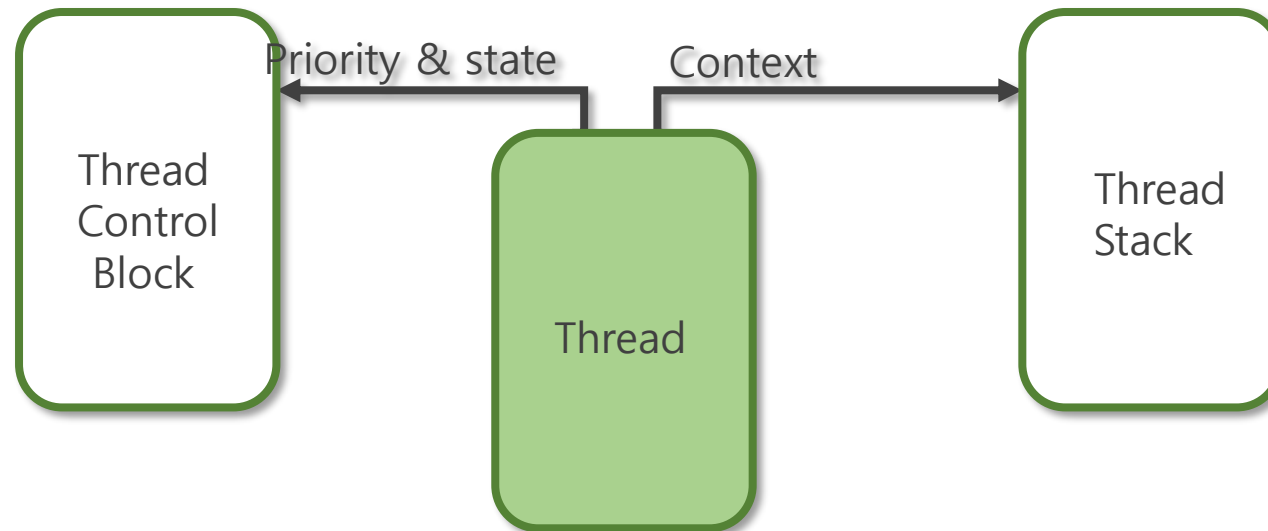
간지

ARM 프로세서 종류, 특징 학습

- ARM 프로세서에서 RTOS 탑재 및 활용
- RTOS 스레드

각 스레드는 컨텍스트 전환 중 데이터를 저장하기 위한 자체 스택을 가짐

스레드 제어 블록(thread control block)은 커널에서 활성 스레드를 관리하는 데 사용됨.



- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS Thread 응용

Main() 함수가 실행될 때 기본적으로 CMSIS-RTOS scheduler가 동작

main() 함수는 첫번째 active thread임

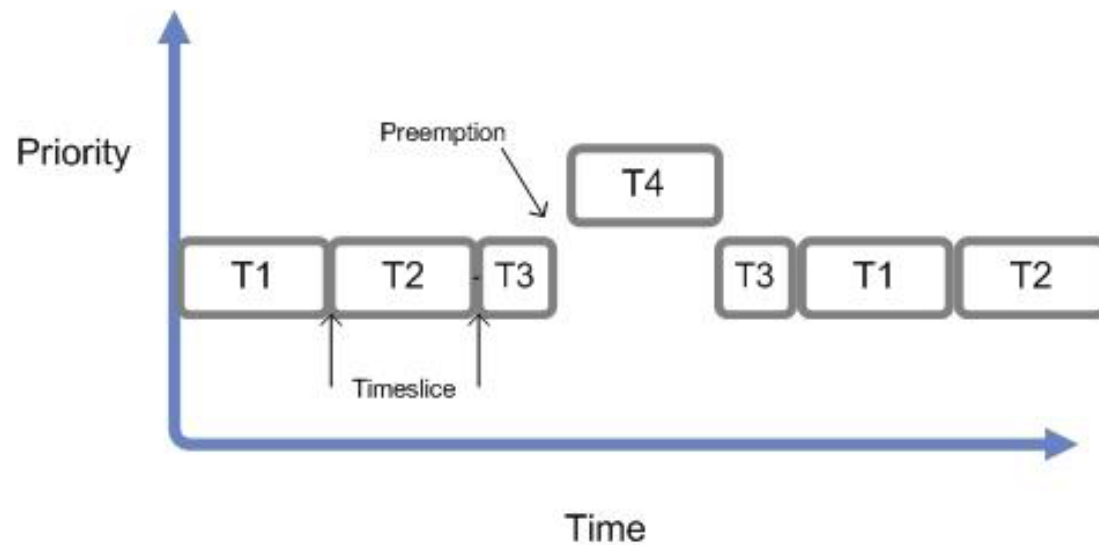
osKernelInitialize() 함수 호출에 의해서 scheduler task를 switching할 수 있음

Init_Thread() 함수에서 추가적인 thread 생성 및 RTOS object 생성

osKernelStart() 함수를 실행함으로써 RTOS scheduler를 재시작

상태	설명
Running	실행중인 스레드 상태
Ready	실행이 준비된 스레드 상태
Wait	특정 Event를 대기중인 Blocked Threads 상태

- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS Thread 응용
- 쓰레드가 생성될 때 default priority 를 할당
Ready 쓰레드가 여러 개이고, 모두 같은 priority를 가질 경우, 이들은 round-robin fashion으로 실행됨
- 동일한 우선순위의 스레드는 라운드 로빈 방식으로 예약됨. 우선순위가 높은 작업은 우선순위가 낮은 작업을 선점하고 '주문 시' 실행 상태에 들어감.



CMSIS-RTOS Priority Levels

osPriorityIdle
osPriorityLow
osPriorityBelowNormal
osPriorityNormal
osPriorityAboveNormal
osPriorityHigh
osPriorityRealTime
osPriorityError

- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS Thread 응용
- Ex 2 및 3 스레드 " 에서 LED 스레드 2의 우선순위를 Normal 이상으로 변경
 osThreadDef(led_thread2, osPriorityAboveNormal, 1, 0);
 osThreadDef(led_thread1, osPriorityNormal, 1, 0)
- LED_thread1은 Normal인 우선순위로 실행, LED_thread2는 더 높은 우선순위로 실행
 되므로, LED_thread1보다 우선하여 실행됨.
 설상가상으로 led_thread2는 우선순위가 높으므로, 그보다 낮은 우선순위의 LED_thread1 스
 레드보다 우선하여 끝까지 실행됨.

- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS Thread 응용
- osDelay() 함수 사용
- LED 플래셔 프로그램

단순 지연 기능이 osDelay API 호출로 대체되었음.
LED2는 100mS마다, LED1은 500mS마다 토글됨

- Void osDelay (uint32_t millisec)

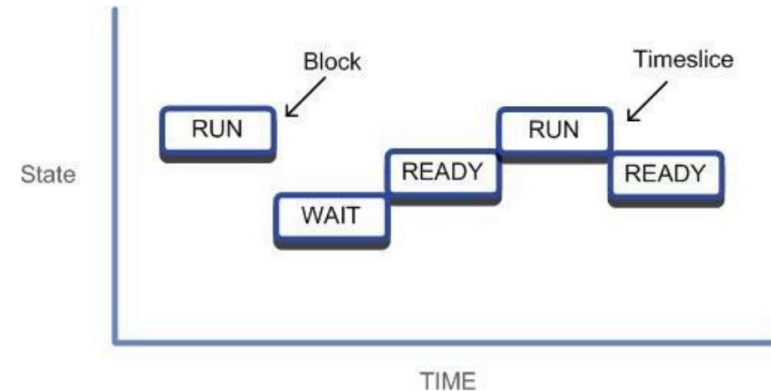
이 호출은 지정된 밀리초 동안 호출 스레드를
WAIT_DELAY 상태로 전환함.

스케줄러는 READY 상태의 다음 스레드로 실행을 전달함
스레드는 수명 동안 여러 상태를 이동함.

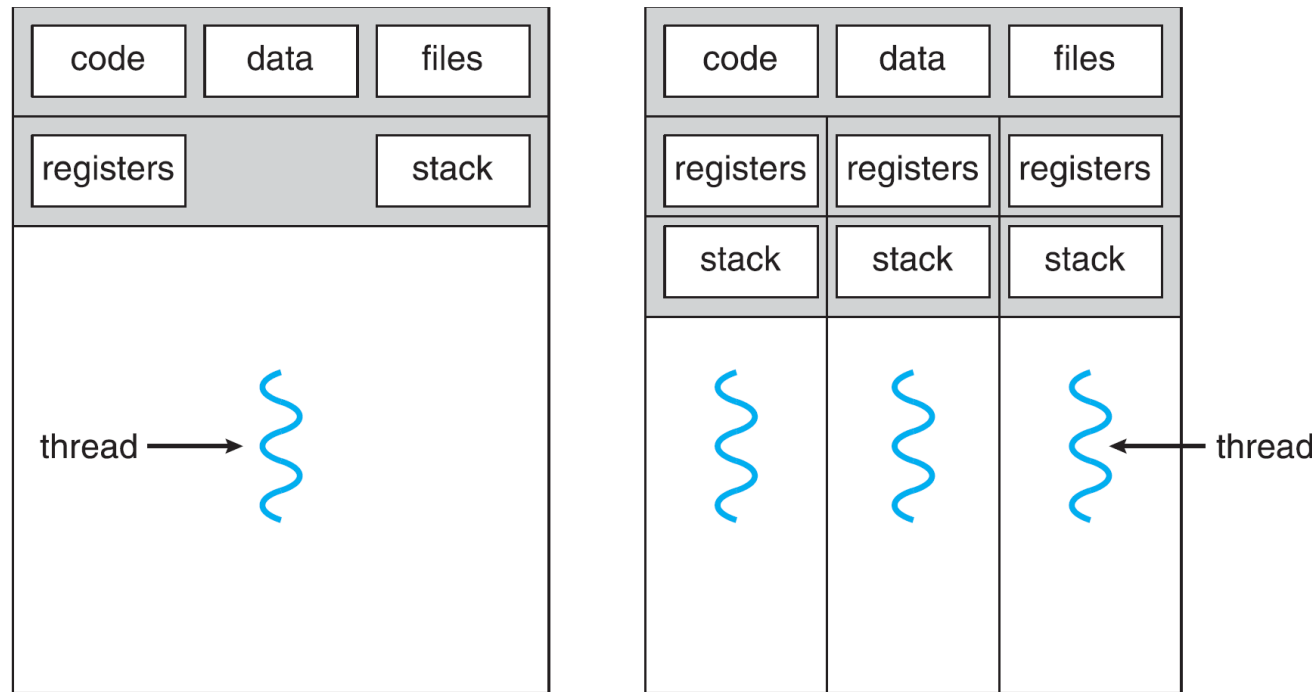
실행 중인 스레드는 osDelay 호출에 의해 차단되어 대기
상태에 들어감.

지연이 완료되면 준비 상태로 이동함.

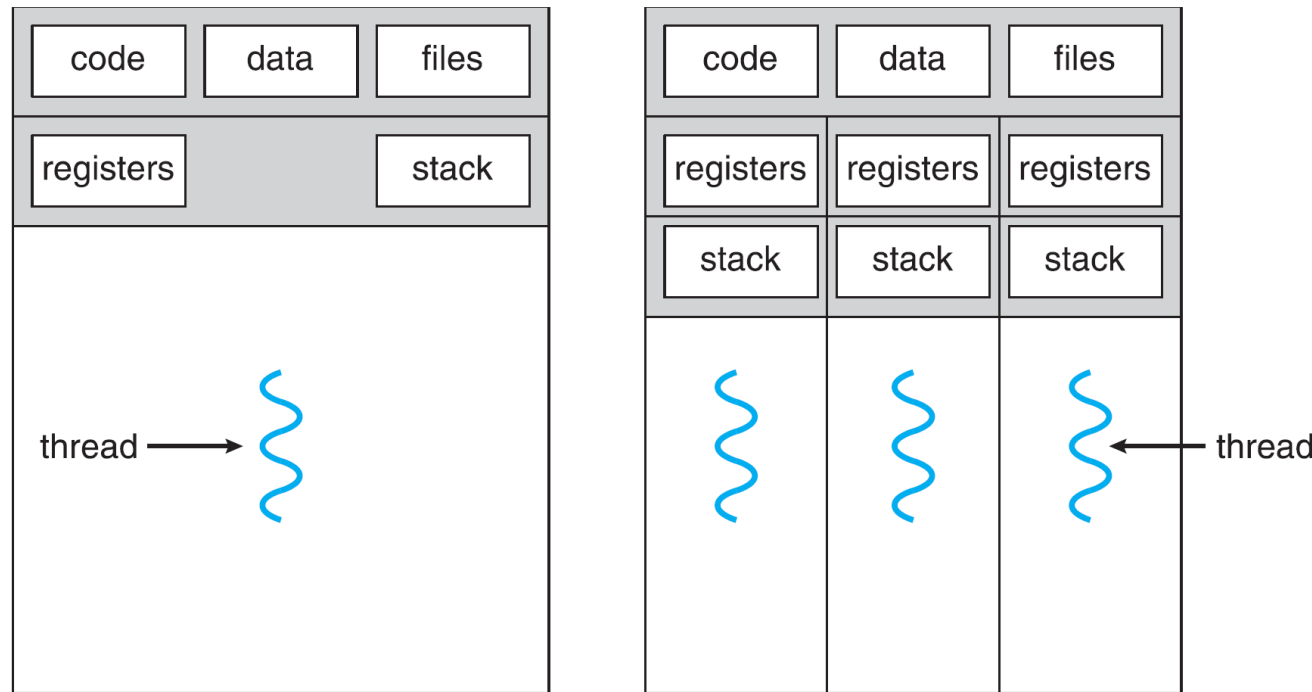
스케줄러는 스레드를 실행 상태로 전환하며, 타임슬라이
스가 완료되면 다시 준비 상태로 이동함.



- ARM 프로세서에서 RTOS 탑재 및 활용
- RTOS 쓰레드
- Each thread has its own stack for saving its data during a context switch. The thread control block is used by the kernel to manage the active thread



- ARM 프로세서에서 RTOS 탑재 및 활용
- RTOS 쓰레드
- Each thread has its own stack for saving its data during a context switch. The thread control block is used by the kernel to manage the active thread



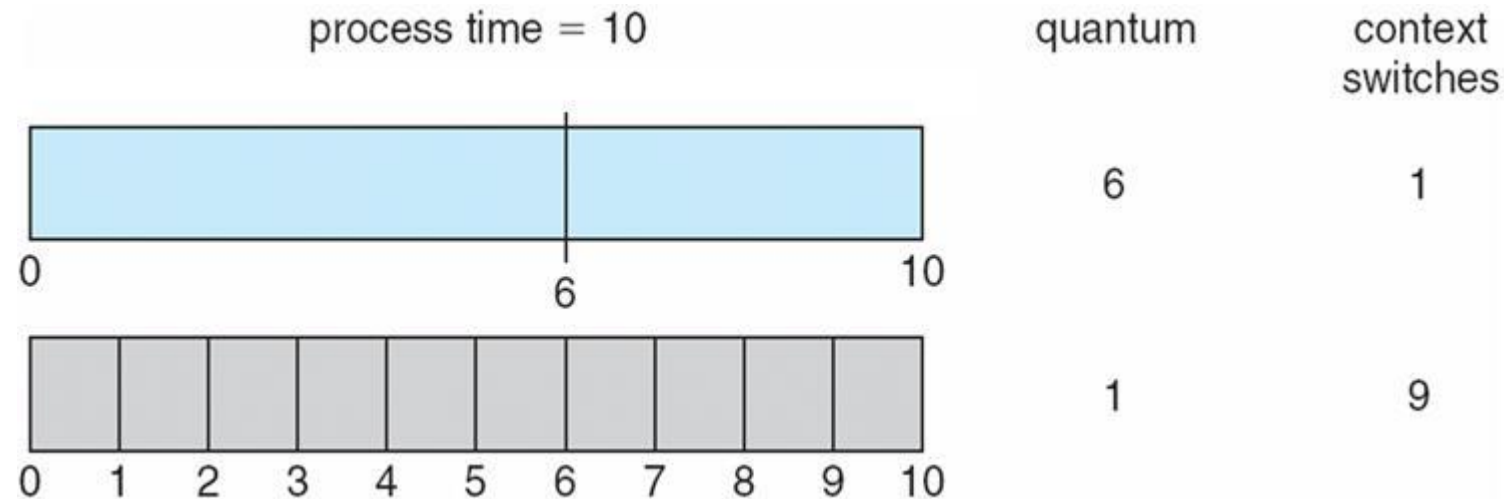
- ARM 프로세서에서 RTOS 탑재 및 활용
- Round Robin scheduling

각 프로세스는 보통 10-100밀리초 단위의 CPU 시간(시간 양자)을 얻음

이 시간이 경과하면 프로세스가 선점되어 준비 대기열의 끝에 추가됨.

준비 대기열에 n 개의 프로세스가 있고 시간 양자가 q 인 경우 각 프로세스는 한 번에 최대 q 시간 단위로 CPU 시간의 $1/n$ 을 얻음.

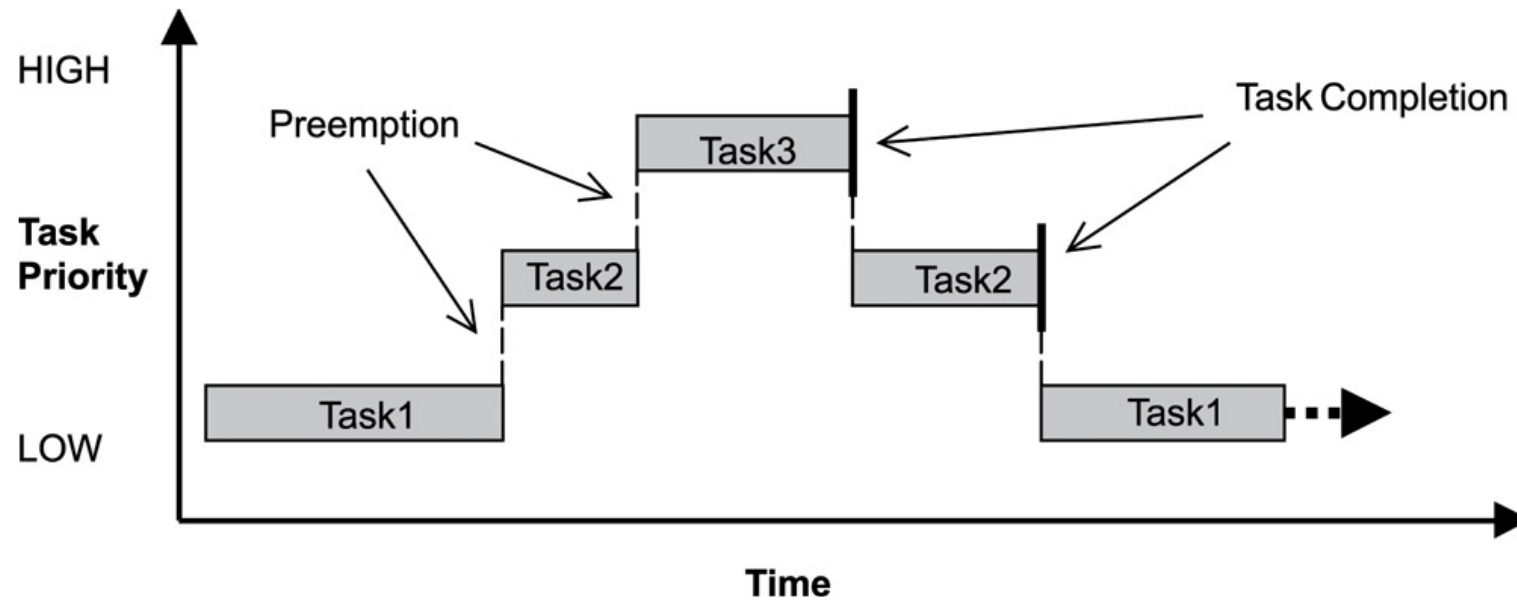
결과적으로, 한 프로세스는 최대 $(n-1)q$ 시간 단위를 초과하지 않는 범위에서 자신이 CPU를 할당받을 수 있음.



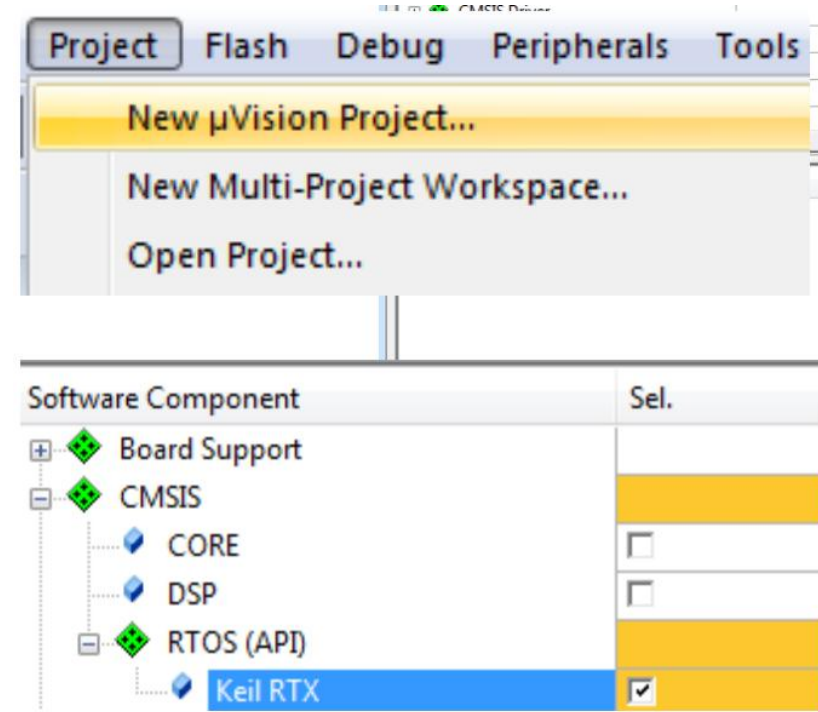
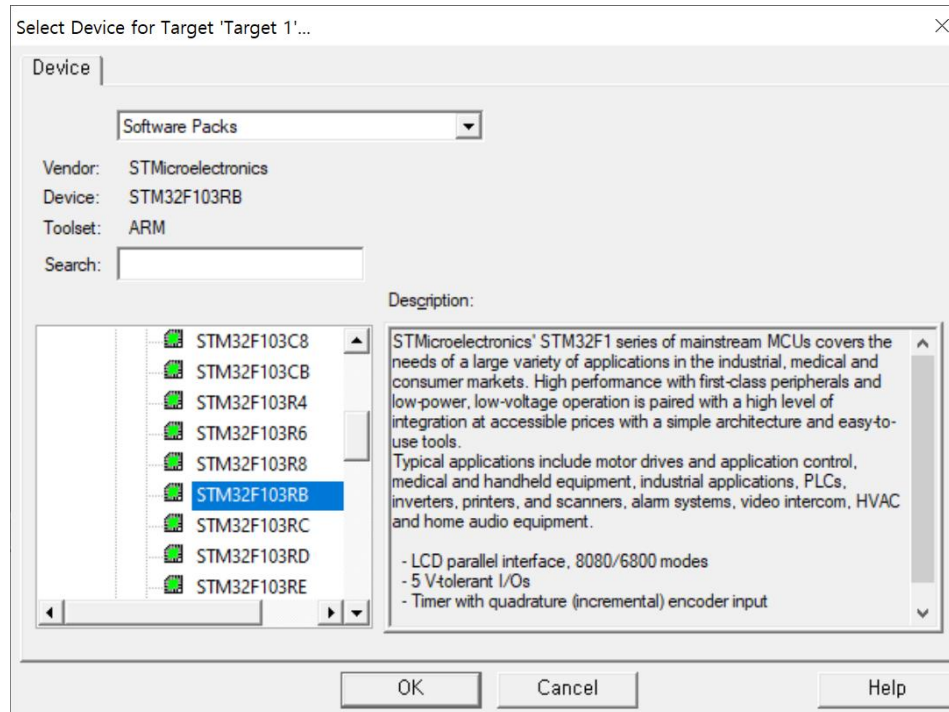
- ARM 프로세서에서 RTOS 탑재 및 활용
- Priority-Based preemption Scheduling

연성 실시간 기능을 제공하는 것에 불과함.

경성 실시간 시스템에서는 태스크가 마감시간 내에 확실히 수행되는 것을 보장해야만 하므로
부가적인 스케줄링 기법이 필요

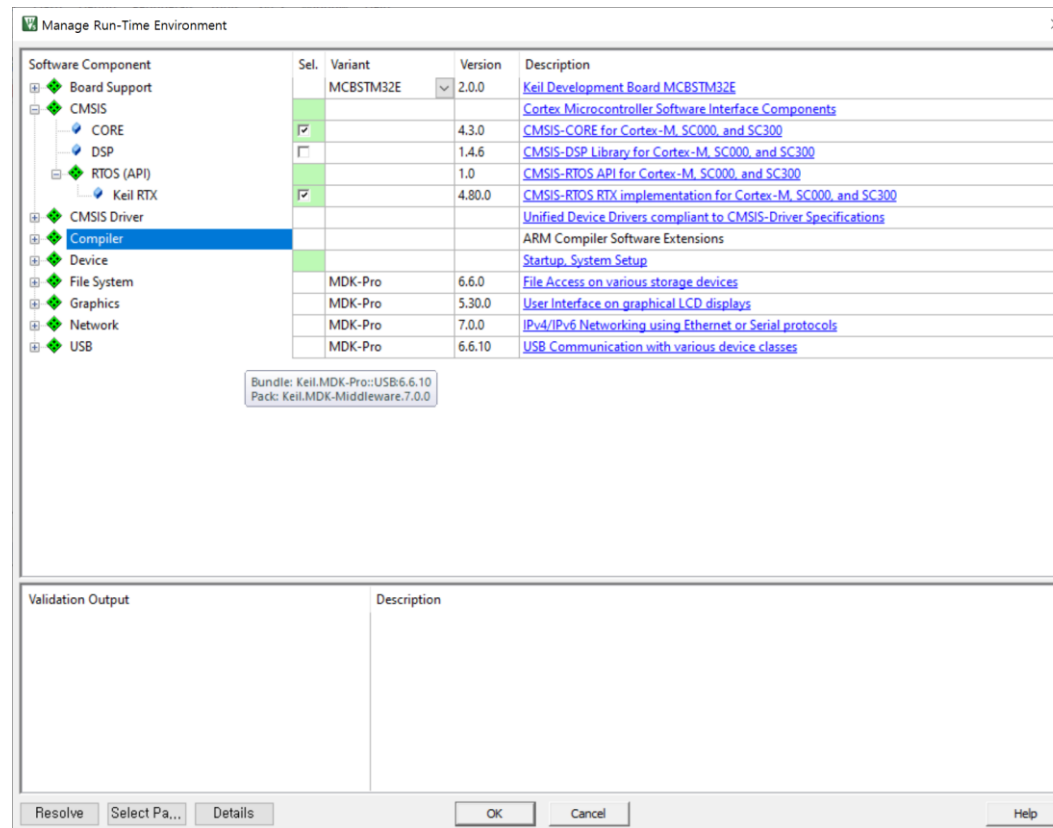


- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS 기본 프로젝트 생성



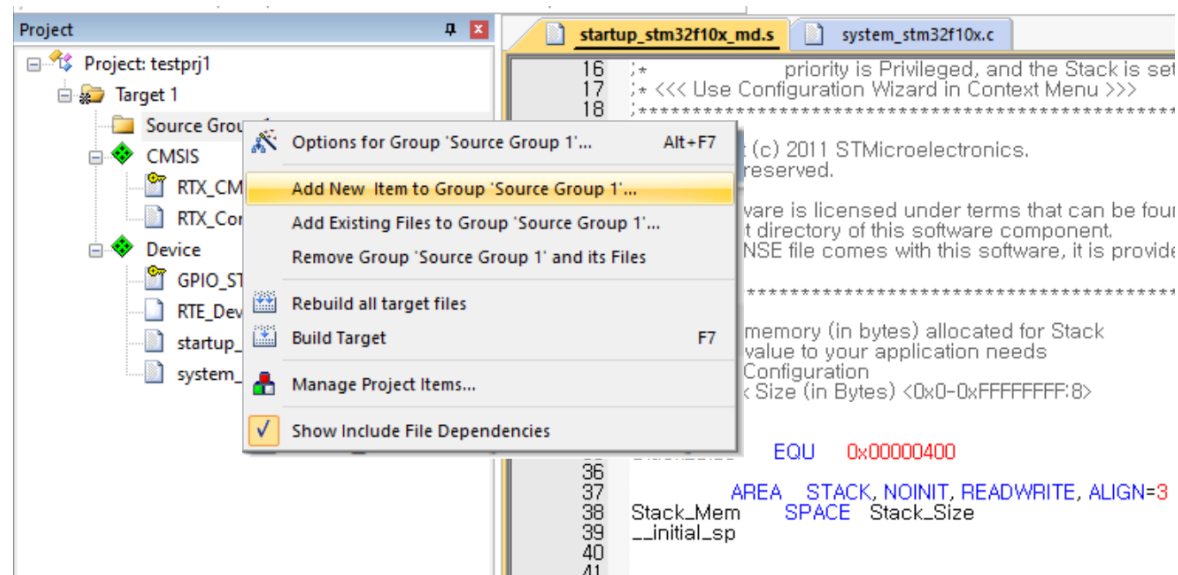
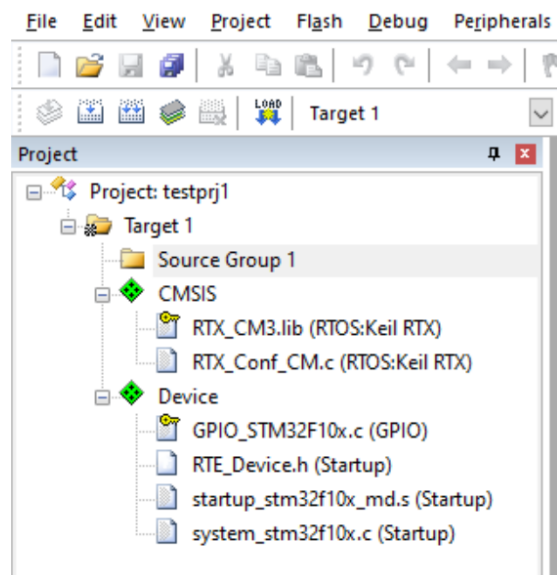
- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS 기본 프로젝트 생성

RTE의 왼쪽 하단 모서리에 있는 "해결" 버튼을 통해 누락된 구성 요소를 추가하는 방법도 가능

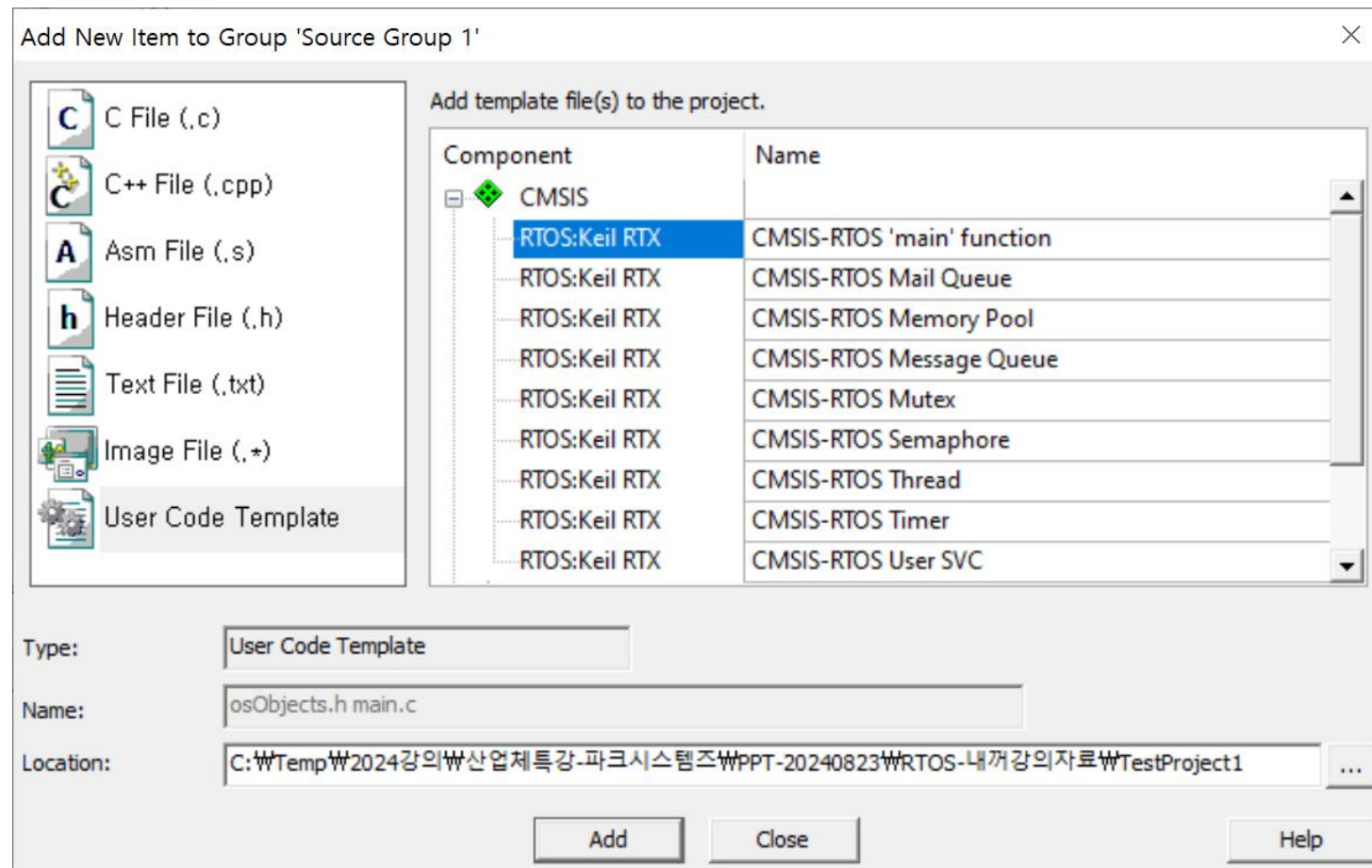


- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS 기본 프로젝트 생성

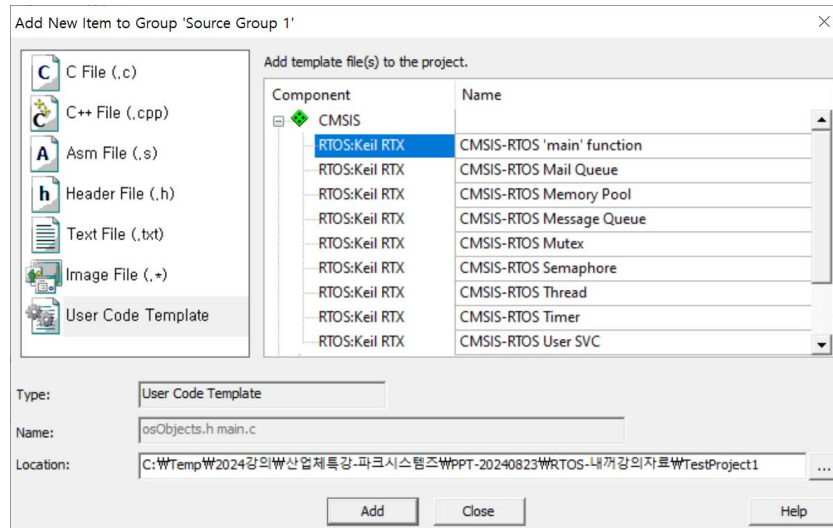
파일	설명
Startup_STM32F10x_md.s	벡터테이블을 포함한 어셈블리 코드
System_STM32F10x.c	시스템 및 주변장치 초기화 코드, Clock Tree, PLL, 외부 메모리 인터페이스 등에 관한 설정
RTE_Device.h	Configures the pin multiplex
RTX_Conf_CM.c	Configures Keil RTX



- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS 기본 프로젝트 생성



- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS 기본 프로젝트 생성



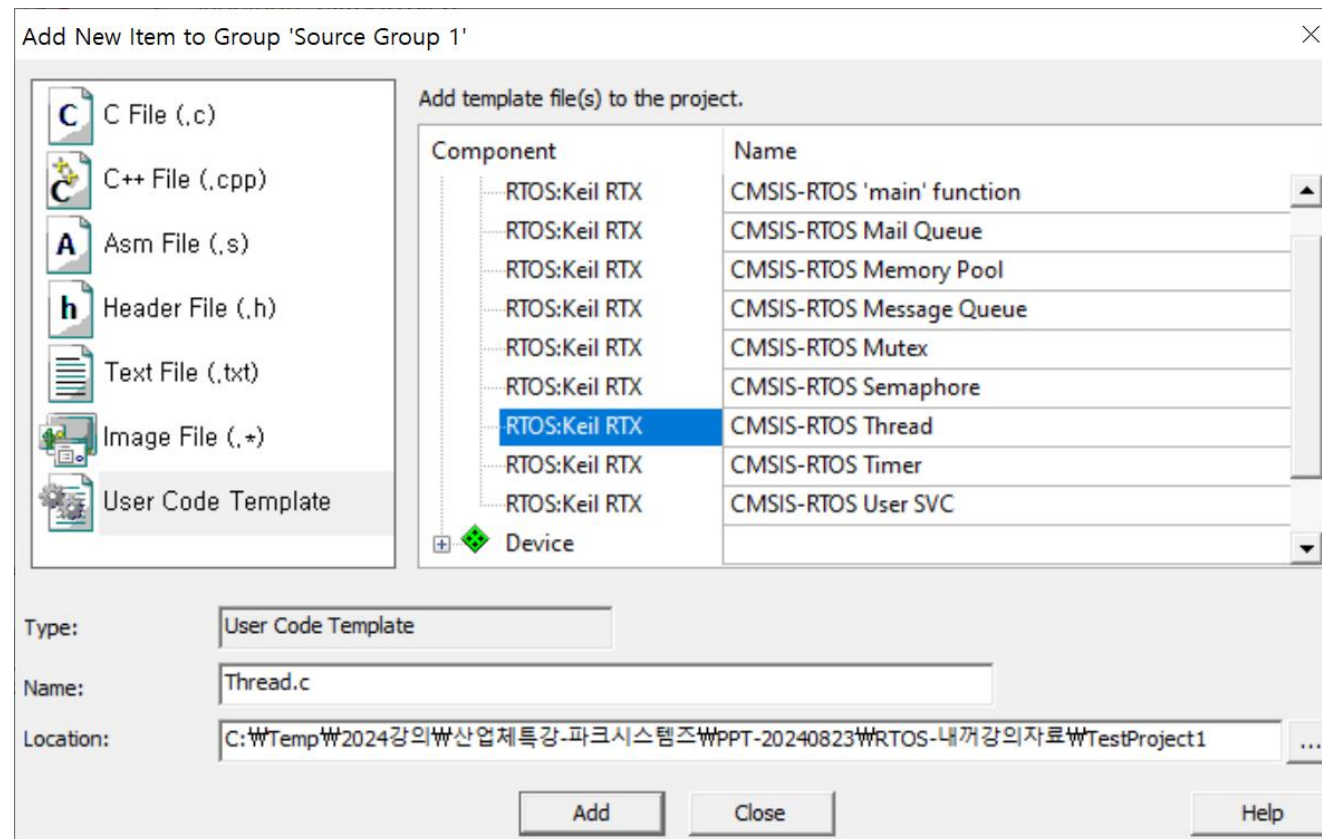
```
#define osObjectsPublic           // define objects in main
module                           module
#include "cmsis_os.h"

int main (void) {
    osKernelInitialize ();        // initialize CMSIS-RTOS

    // initialize peripherals here
    // create 'thread' functions that start executing,
    // example: tid_name = osThreadCreate (osThread(name),
    NULL);

    osKernelStart ();             // start thread execution
}
```

- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS Thread 추가 생성



- ARM 프로세서에서 RTOS 탑재 및 활용4
- RTX RTOS Thread 추가 생성

```
63 osThreadId main_ID,led_ID1,led_ID2;  
64 osThreadDef(led_thread2, osPriorityNormal, 1, 0);  
65 osThreadDef(led_thread1, osPriorityNormal, 1, 0);  
  
71 int main (void)  
72 {  
73     osKernelInitialize ();           // initialize CMSIS-RTOS  
74  
75     LED_Initialize ();  
76     led_ID2 = osThreadCreate(osThread(led_thread2), NULL);  
77     led_ID1 = osThreadCreate(osThread(led_thread1), NULL);  
78  
79     osKernelStart ();               // start thread execution  
80     while(1)  
81     {  
82     };  
83 }  
84 }
```

```
33 void led_thread1 (void const *argument)  
34 {  
35  
36     for (;;)   
37     {  
38         LED_On(0);  
39         delay(500);  
40         LED_Off(0);  
41         delay(500);  
42     }  
43 }  
44  
45 void led_thread2 (void const *argument)  
46 {  
47     for (;;)   
48     {  
49         LED_On(2);  
50         delay(500);  
51         LED_Off(2);  
52         delay(500);  
53     }  
54 }
```


- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS Thread 추가 생성

ID	Name	Priority	State	Delay	Event Value	Event Mask	Stack Usage
1	osTimerThread	High	Wait_MBX				32%
3	led_thread2	Normal	Running				0%
4	led_thread1	Normal	Ready				32%
255	os_idle_demon	None	Ready				

- ARM 프로세서에서 RTOS 탑재 및 활용

- RTX RTOS Thread 응용

- 세마포어(semaphore)

프로세스가 활동을 동기화할 수 있는 더 정교한 방법(무텍스 잠금 장치보다)을 제공하는 동기화 도구.

두 개 이상의 스레드 간의 활동 동기화 방법

세마포어는 프로그램 리소스에 대한 액세스를 제어하는 데 유용함

스레드가 리소스에 액세스하기 전에 토큰을 획득해야 하고, 사용할 수 있는 리소스가 없으면 대기하여야 함.

리소스가 완료되면 토큰을 반환함.

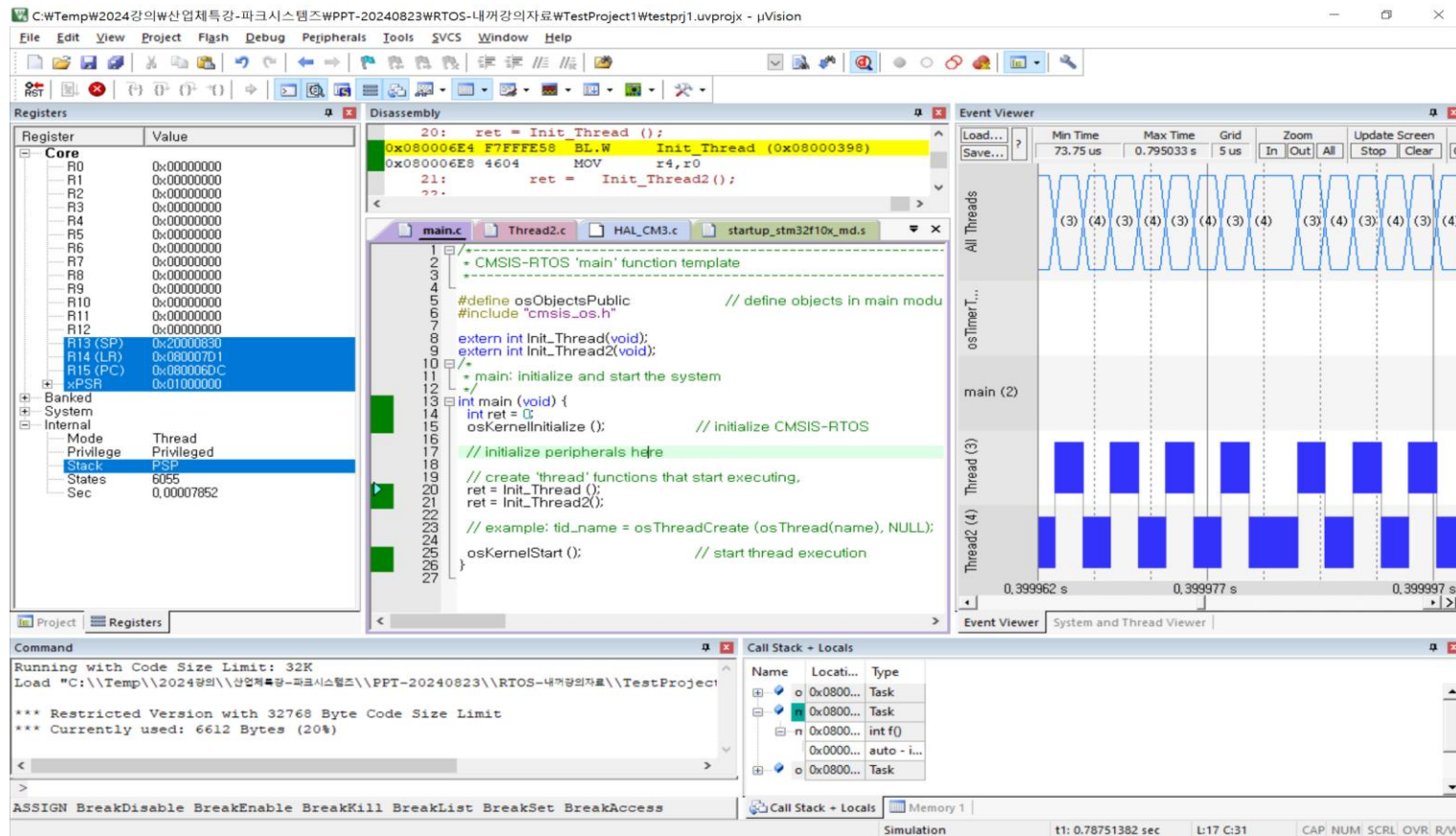
- 임계 섹션(critical section)

각 프로세스에는 critical section에 해당하는 code segment가 있습니다

이러한 critical section에서 공통 변수 변경, 테이블 업데이트, 파일 작성 등을 할 수 있음

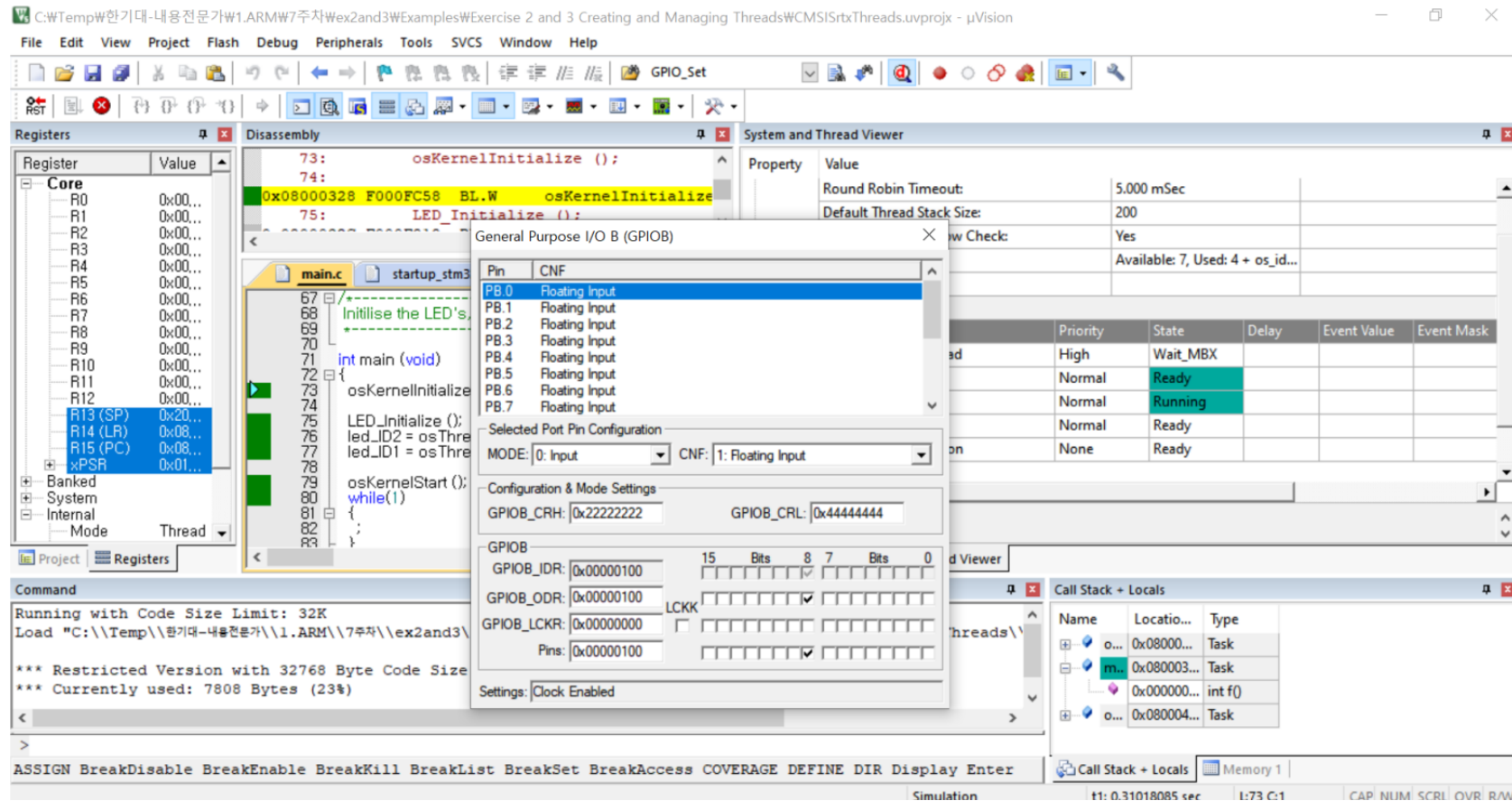
한 프로세스가 critical section에 있을 때, 다른 프로세스는 더 이상 임계 섹션에 들어올 수 없음

- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS Thread 응용



- Open the Debug -> OS Support -> Event Viewer

- ARM 프로세서에서 RTOS 탑재 및 활용
- RTX RTOS Thread 응용



Debug -> OS Support -> System and Thread Viewer

- Round Robin과 Priority-based Preemptive 스케줄링은 어떻게 다른가요?

Round Robin은 같은 우선순위를 가진 태스크들이 CPU 시간을 공평하게 나누어 쓰는 방식이에요. 반면, Priority-based Preemptive는 우선순위가 높은 태스크가 언제든지 낮은 태스크를 선점(preempt) 할 수 있습니다. 즉, RTOS는 이런 방식으로 정확한 시간 제어와 실시간 응답성을 유지합니다.

- RTOS가 일반적인 운영체제(OS)와 다른 점이 무엇인가요?

일반 OS는 성능이나 사용자 편의성 위주로 설계되지만, RTOS(Real-Time Operating System)는 “정해진 시간 안에 작업이 반드시 끝나야 하는” 실시간성이 핵심이에요. 즉, 단순히 빠르게 처리하는 것이 아니라, 예측 가능한 시간 안에 반드시 처리하는 것이 RTOS의 본질입니다.